

Modelling and Simulation

Lecture 2

1. Introduction to high performance computing

Why do we need programming in this course?

- We want to represent nature through complex mathematical models.
- These models are too complex to be solved analytically.
- Either they do not possess analytical solutions or it would take too long to compute it.
- Numerical methods exist to approximate the mathematical models used to represent nature.
- Computers are very efficient in doing large amount of computations.
- We need an efficient way to make the computer use the numerical methods.

Computers are fast but... we always want more

For a computer model to be satisfying it must...

- Represent with good accuracy the process it is supposed to model.
- Give a solution in a reasonable amount of time.

For a program to be faster one can...

- Wait for the hardware to become faster (Moore's law).
- Optimize the algorithm (bubble sort, quick sort).
- Optimize the way algorithm implemented.

2. Concepts of code optimization

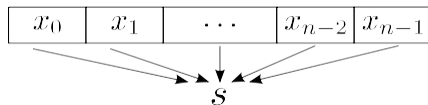
Computers are fast but... we always want more

Example: sum of the numbers stored in an array.

Mathematical representation

$$A = \{x_i\}_{i=0}^{n-1}, x_i \in \mathbf{N}, s = \sum_{i=0}^{n-1} x_i$$

Graphical representation

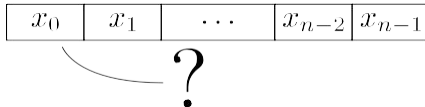
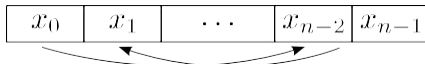


How do we implement this on a computer?

Computers are fast but... we always want more

Performed in 3 different ways with Python (there are more)

- The array is read linearly.
- The array is read randomly.
- The sum is performed with sum.



Benchmark: Sum of $n = 10^6$ integers

Efficiency depends on the algorithm.

	linear	random	sum
time [s]	0.0594	0.270	0.00654
t/t_{sum}	9.08	41.3	1

Efficiency depends on the programming language

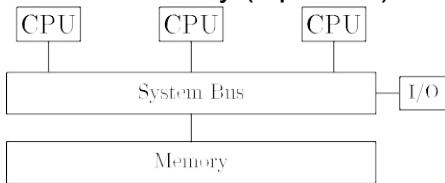
	Python 3 (sum)	NumPy	C++ (linear)
time [s]	0.00654	0.000634	0.000478
t/t_{C++}	13.7	1.3264	1

By coding naively C++ is 100 times faster than Python but with NumPy the performance is equivalent.

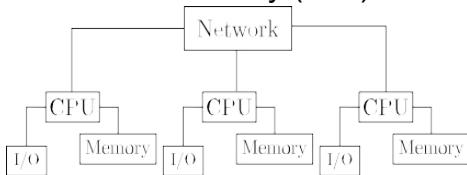
3. Concepts of parallelism

A computer is fast... many computers together are faster

Shared memory (OpenMP)



Distributed memory (MPI)



Performance

- One modern processor $\sim 2\text{-}3 \text{ Tflops} = 2\text{-}3 \cdot 10^{12} \text{ flops}$
- One modern GPU $\sim 82.6 \text{ Tflops} = 82.6 \cdot 10^{12} \text{ flops}$
- The fastest supercomputer (as of 2024) $\sim 1 \text{ Exaflops} = 10^{18} \text{ flops}$.

A computer is fast... many computers together are faster

Not all problems are parallelizable

Parallelizable problem

Can be decomposed into pieces that can be executed simultaneously.

Non-parallelizable problem

Fibonacci series computation

$$F(i) = F(i - 1) + F(i - 2),$$

$$F(1) = 1, F(2) = 1.$$

4. Palabos, a parallel lattice Boltzmann solver

The Palabos library

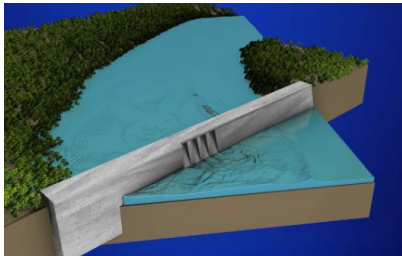
Palabos, <http://www.palabos.org>

- C++ parallel open-source library.
- Parallelism completely transparent.

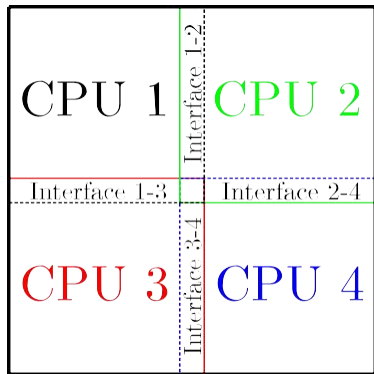
Example:



- Fluid flow solver based on LBM.
- Multi-physics modules.



Parallelism in Palabos: Multi-Block approach



- Spatial domain cut into many blocks.
- Blocks are dispatched on different CPUs.
- Communication is done through the interfaces.
- Efficient if communication $\ll$ computation and the load on each processor is well balanced.